

QA evidence — server artifact signature consistency (verified == served)

Revision: 1 **Last modified:** 2026-07-10T06:44:00Z **Run-id:** 20260710-server-signature-consistency **Scope:** server/ Go module (main repo). No submodule touched. **Anchor:** §11.4.6 (verify-as-FACT), §11.4.115 (RED-baseline + polarity switch), §11.4.107/§11.4.108 (served-to-device runtime signature), §11.4.83 (QA transcript).

Defect (verified as FACT, not surfaced-guess)

The server verified one detached signature against the payload but stored + served a *different*, unverified signature to devices. Chain, all cited file:line:

- `internal/api/handlers_artifact.go:124` — `sig = resolveSignature(form, meta.Signature).resolveSignature (:274)` PREFERS an uploaded signature form-part over `metadata.signature`, so `sig` is the signature actually verified.
- `internal/api/handlers_artifact.go:147` — `in.Signature = sig` → the ONLY signature cryptographically checked (`ota-artifact-validator/pipeline.go:73 ValidateSignature(digest, in.PublicKey, in.Signature)`).
- Validator NEVER cross-checks `Meta.Signature` vs `Input.Signature`; S6 `ValidateMetadata` requires `meta.signature` merely non-empty + non-blank (`ota-protocol/validate.go:79-84`).
- `internal/api/handlers_artifact.go:177` (pre-fix) — `art.Signature = meta.Signature` → the STORED value is the unverified `metadata.signature`.
- `internal/api/handlers_client.go:81` — `UpdateAvailable.Signature = art.Signature` → that stored, possibly-divergent value is SERVED to the device.

Reachable + harmful scenario: an upload whose verified signature form-part differs from `metadata.signature` is ACCEPTED (201, `Verified:true`), yet the device is served a signature that does not match the payload → device signature check fails → cannot install, while the server reported the artifact valid/published. §11.4 “validation passes but broken for the end user”.

Fix (server layer only; single source of truth)

`internal/api/handlers_artifact.go:186` — store `base64.StdEncoding.EncodeToString(sig)` (the base64 of the actually-verified signature) instead of `meta.Signature`. Cannot false-reject a legitimate publish: when the signature was supplied only via `metadata.signature` (canonical std base64), re-encoding the decoded bytes yields the identical string; when a signature form-part was verified, the stored value becomes exactly that verified signature. The decoupled `ota-artifact-validator` library was NOT touched (§11.4.28 — it is encoding-agnostic; the server owns base64).

RED baseline (§11.4.115) — captured on the PRE-FIX artifact

Test: `TestArtifactUploadServesVerifiedSignature` drives the REAL device-facing `/client/update` path and asserts the served `UpdateAvailable.Signature` equals the verified signature. Default mode = GREEN guard (assert defect absent). Verbatim pre-fix failing output:

```
=== RUN    TestArtifactUploadServesVerifiedSignature
    handlers_artifact_signature_consistency_test.go:142: device
served a signature that does NOT match the verified one: server
verified "rJF8+ba04/KEGo06cVknGTGod6toEcFWZcZRFWSZr0moqVr51/
TW01081nmTYuzTJpqekgBdm7XvYQL40bmNCQ==" against the payload but
serves
"VtZq3QjIMHvWkcgecAoZvTUIpg9T3kPOBSEuRSY1l798oVHWwgYSY65CPfE3ddSbTk
to the device -> the device receives a signature that does not
match the payload and fails to install while the server reported
the artifact valid/published
--- FAIL: TestArtifactUploadServesVerifiedSignature (0.00s)
FAIL
```

Polarity proof on PRE-FIX code — RED_MODE=1 (assert defect PRESENT) PASSED, confirming the defect is genuinely reproduced (served == divergent metadata.signature):

```
=== RUN    TestArtifactUploadServesVerifiedSignature
--- PASS: TestArtifactUploadServesVerifiedSignature (0.00s)
PASS
```

GREEN (post-fix)

- Default GREEN guard PASSES — see green_guard_postfix.log.
- RED_MODE=1 (defect-present) now FAILS — see redmode_postfix_defect_absent.log — a valid §1.1 pair: the same source guards the invariant, and its mutation (RED_MODE) flips the verdict.

Full verification sweep (post-fix)

- go build ./... → exit 0; go vet ./... → exit 0; gofmt -l internal/api → clean. See build_vet_gofmt.log.
- go test -race ./internal/api/... → ok (all existing upload+serve tests still pass). See race_internal_api.log.
- go test ./... (whole module) → all packages ok (Postgres integration tests skip without a DB).

Files changed (relative to server/)

- internal/api/handlers_artifact.go — fix at the accept-path store.Artifact.Signature.
- internal/api/handlers_artifact_signature_consistency_test.go — NEW §11.4.115 RED test / permanent regression guard.
- docs/qa/20260710-server-signature-consistency/ — this evidence.